

Formalização da correção do algoritmo Selection Sort

Erik Cojho 23102133

Angel Flavius 241038110

Lucas Santana 211060666

14 de julho de 2026

Require Import Arith List Lia.

Require Import Sorted.

Require Import Permutation.

Import ListNotations.

1 Motivação

O Selection Sort ordena uma lista repetindo o seguinte procedimento: escolhe o menor elemento da parte ainda não ordenada da lista, coloca esse elemento na primeira posição dessa parte e repete o processo sobre o restante da lista.

Só que notamos que para provar formalmente que o algoritmo está correto, não bastava mostrar que a saída está em ordem crescente. Também precisamos garantir que nenhum elemento foi perdido, duplicado ou criado.

Pra isso, precisamos garantir que está ordenada e a saída está preservando os elementos da lista original

- **Sorted** *le* (*ss l*): Onde a lista (*l*) vai representar um resultado obtido pelo Selection Sort (*ss*) e *le* está ordenada usando a relação menor ou igual, e o resultado vai tá em ordem crescente.
- **Permutation** *l* (*ss l*): Tanto a lista original e a resultante possuem os mesmos elementos, a ordem pode ter mudado, mas nenhum valor será perdido, criado ou duplicado.

2 Um elemento é menor ou igual a todos os elementos da lista

Estabelecemos quais propriedades precisaríamos provar para a corretude do algoritmo, então pensamos em como representar a ideia do menor elemento. No Selection Sort, a cada etapa precisamos escolher um valor que seja menor ou igual à todos os outros elementos da parte da lista que ainda não foi ordenada. Então criamos essa função, *le_all* (less than or equal to all), ou seja, menor ou igual a todos. Ela recebe dois argumentos: *x* : **nat**, que *x* representa um número natural e *l* : **list nat**, onde *l* representa uma lista de números naturais. O resultado da definição possui o tipo **Prop**, então *le_all*(*x*, *l*) não produzirá uma nova lista, mas sim fará uma proposição lógica, que significa que uma afirmação que pode ter como valor: verdade, ou podemos identificar ela como falsa. A ideia da nossa definição é considerar qualquer elemento *y* que pertencer a uma lista *l*. Todavia, precisaríamos provar que *x* é menor ou igual *y*. Essa definição vai ser importante nas próximas etapas da prova, já que irá garantir que o elemento escolhido pela função da seleção é realmente o menor valor da lista.

Definition *le_all* (*x* : **nat**) (*l* : **list nat**) : **Prop** :=
 $\forall y, \text{In } y \ l \rightarrow x \leq y.$

3 Seleção do menor elemento

Agora que definimos `le_all`, o próximo passo seria pensar em como o selection sort faria para encontrar esse elemento quando for executado. Inicialmente, a primeira ideia seria utilizar uma função que receberia uma lista e devolveria o menor número, mas seria insuficiente e o pensamento estava errado. O selection sort não precisa saber somente qual é o menor valor, ele deveria continuar ordenando até o fim. Dessa forma pensamos que a função, na verdade, deveria devolver duas informações juntas: o menor elemento encontrado e a lista que foi formada pelos demais elementos. Então:

`select x l` considera os elementos da lista $x :: l$.

O resultado vai ser um par (m, r) o qual:

- m é o menor elemento de $x :: l$,
- r seria todos os demais elementos,
- $x :: l$ então é uma permutação de $m :: r$.

Essa propriedade vai garantir que a função `select` não remova, duplique ou crie um elemento, só organizaria e separaria o menor elemento.

Então a função na `select`, definimos então como uma lógica recursiva, e para isso usamos `Fixpoint`. Ela recebe dois argumentos: um número natural x , que funcionaria como um candidato e uma lista l , que terá os valores que ainda vamos analisar.

O resultado possui o tipo `nat × list nat`. Isso é uma função que vai devolver um par formado pelo menor elemento e a lista com os restantes. E para a busca, a função vai analisar a estrutura da lista com `match l with`.

No caso base, seria $l = []$, não tem mais elementos para comparar com o nosso candidato no momento. X será devolvido com o menor elemento e o restante da lista vazia. Já no segundo, a lista tem $h::t$ (cabeça e cauda), e vamos comparar h com o candidato atual que é x . Se for verdadeiro, x continua sendo nosso candidato, a função chama a si mesmo sobre a cauda, mantendo x e devolve um par (m, r) , e como h não foi escolhido para continuar como candidato, ele precisa estar na lista restante, por isso o resultado $(m, h::r)$. Caso seja falso, significa que h é menor x . Nesse caso, h passa a ser o novo candidato de menor elemento e chama a função de novo, e retorna a mesma estrutura de saída do caso verdade. Nesse caso o x não pode ser descartado, então é anexado à lista do restantes.

Dessa forma à cada etapa, o menor valor entre x e h continua sendo comparado com os próximos elementos, enquanto o outro valor é colocado na lista restante. O resultado da recursão m será o menor elemento e r possuirá o resto da lista.

```
Fixpoint select (x : nat) (l : list nat) : nat × list nat :=
  match l with
  | [] => (x, [])
  | h :: t =>
    if x <= h then
      let '(m, r) := select x t in
      (m, h :: r)
    else
      let '(m, r) := select h t in
      (m, x :: r)
  end.
```

4 Especificação da função `select`

Este lema é o mais importante da primeira parte. Depois de definir `select`, já podemos executar a função, ela receberá um candidato x , analisará l e devolverá um par (m, r) . Observando que a função ao devolver o par

não é o bastante para utilizarmos o `select`, precisamos demonstrar formalmente que o resultado produzido por ela possui o comportamento que esperamos. Por isso criamos o lema `select_spec`. Spec vem de specification, esse lema funcionará como um verificador da função: sempre que `select x l` devolve o par (m, r) , algumas propriedades devem ser obrigatoriamente verdadeiras.

A hipótese:

$[select\ x\ l = (m, r)]$

Indica que ao executar `select` sobre os elementos de $x::l$, o elemento que selecionamos como x e l são respectivamente m e a lista restante r . A partir dessa hipótese, provaremos 3 propriedades, a primeira é:

$[Permutation\ (x :: l)\ (m :: r)]$

Ela vai garanti que a lista de entrada e a lista que foi produzida possui os mesmos elementos. A função altera posições.

A segunda é:

$[all\ m\ (x :: l)]$

Vai garanti que m é menor ou igual à todos os elementos considerados pela função, então m representa o menor elemento de $x::l$

A terceira é:

$[length\ r = length\ l]$

Depois que o único elemento é separado como mínimo, a lista possui a forma $x :: l$. O restante r deve possuir o mesmo tamanho 1. Assim confirmamos que um elemento foi separado e nenhum outro elemento desapareceu durante o processo. Essas três propriedades juntas descrevem completamente o comportamento da função `select` e serão usadas mais adiante na prova do Selection Sort.

Lemma `select_spec` :

$\forall\ x\ l\ m\ r,$
 $select\ x\ l = (m, r) \rightarrow$
 $Permutation\ (x :: l)\ (m :: r) \wedge$
 $le_all\ m\ (x :: l) \wedge$
 $length\ r = length\ l.$

Começamos introduzindo x e l . Não colocamos m, r e a hipótese sobre o resultado de `select`, por que ainda precisamos decidir a estrutura que faremos a indução. A função `select` é recursiva sobre a lista l . Então, nossa prova também será feita por indução. Durante a execução da função, o candidato ao menor elemento pode variar. Na chamada recursiva, podemos executar `select x t`, mas em outra podemos executar `select h t`. Se mantivéssemos x fixo no contexto antes da indução, a hipótese de indução poderia ficar específica demais e não poderia ser aplicada quando o candidato mudasse para h . O comando `revert x` devolve x ao nosso objetivo. Garantindo com que a hipótese de indução será válida para qualquer candidato ao menor elemento. Nossa indução sobre l criará dois casos, sendo o primeiro em que $l = []$ e o passo indutivo, em que $l = h :: t$. Depois da divisão dos casos, colocaremos novamente o x , o menor elemento m , a lista restante r e a hipótese `Hselect`. Nosso objetivo possui três propriedades conectadas por $\wedge$. O primeiro `split` separa a propriedade de permutação das outras duas propriedades. Precisamos provar que $x :: []$ é uma permutação de $x :: []$. Como as duas listas são iguais, usamos a reflexividade da relação de permutação. Ainda restam duas propriedades conectadas por $\wedge$:

- $le_all\ x\ [x];$
- $length\ [] = length\ [].$

Um novo `split` cria um objetivo para cada uma delas. Para provar $le_all\ x\ [x]$, abrimos a definição de `le_all`. Agora precisamos considerar um elemento qualquer y e uma hipótese `Hy` afirmando que y pertence à lista $x :: []$. A simplificação transforma a hipótese de pertencimento em duas possibilidades, sendo elas: $y = x$; e y pertence à lista vazia. Na primeira possibilidade, sabemos que $y = x$. Substituímos y por x e precisamos provar $x \leq x$. A segunda possibilidade afirma que y pertence à lista vazia. Isso é impossível, portanto encerramos o caso por contradição. Por fim, precisamos provar que o tamanho da lista restante vazia é igual ao tamanho da lista original vazia. Os dois lados são exatamente iguais. O comando `destruct` divide a nossa prova em duas partes:

- Se $x \leq h$ for verdade (*true*);
- Se $x \leq h$ for mentira (*false*).

A gente guarda o resultado desse teste na hipótese *Hxh*. Assim fechamos o caso base e as duas ramificações do if. Provamos com sucesso que sempre que *select* devolve (m, r) , as propriedades da especificação valem.

5 Consequências menores da especificação

Depois que provamos o *select_spec*, temos a descrição completa de como a função *select* funciona.

$[select\ x\ l = (m, r)]$

Teremos três informações:

- Que $x :: l$ é uma permutação de $m :: r$;
- Que m é menor ou igual a todo mundo em $x :: l$;
- Que o tamanho do resto r é igual ao tamanho de l .

Só que nem sempre precisaremos usar essas três coisas ao mesmo tempo nas provas. Às vezes só queremos saber que os elementos foram mantidos com a permutação, outras vezes só precisamos do mínimo, ou só do tamanho. Para não ter que ficar abrindo o *select_spec* e limpando o que não importa toda hora, criamos três lemas menores. Cada um deles puxa só uma parte do que o *select_spec* garante.

5.1 Preservação dos elementos

O lema *select_perm* serve para isolar a primeira parte do *select_spec*. O objetivo dele é provar que, se o *select* pegou uma lista $x :: l$ e devolveu (m, r) , os elementos continuam sendo exatamente os mesmos no final. A ordem até pode mudar, mas ninguém some, duplica ou é criado do nada.

Lemma *select_perm* :

$\forall x\ l\ m\ r,$
 $select\ x\ l = (m, r) \rightarrow$
 $Permutation\ (x :: l)\ (m :: r).$

5.2 O elemento selecionado é realmente o menor de todos

O lema *select_minimum* faz a mesma coisa, mas isola a segunda propriedade ele garante que o m que o *select* devolve é de fato menor ou igual a qualquer elemento da lista original.

Lemma *select_minimum* :

$\forall x\ l\ m\ r,$
 $select\ x\ l = (m, r) \rightarrow$
 $le_all\ m\ (x :: l).$

5.3 Tamanho da lista restante

O lema *select_length* isola a última parte e nos mostra que depois de tirar o menor elemento, a lista que sobra (r) tem o mesmo tamanho da cauda original (l) . Isso prova que a gente tirou exatamente um elemento e não perdeu nada no caminho.

Lemma *select_length* :

$\forall x\ l\ m\ r,$
 $select\ x\ l = (m, r) \rightarrow$
 $length\ r = length\ l.$

6 O mínimo também é menor ou igual aos elementos restantes

A nosso lemma dirá que o m é menor ou igual a todo mundo da lista original $x :: l$. Só que no Selection Sort, depois que colocamos esse m na primeira posição, o algoritmo vai continuar ordenando só a lista que restou (r).

Por isso, precisamos de uma prova que garanta que o m também é menor ou igual a todos da lista restante r ($\text{le_all } m \ r$). Para provar isso, vamos usar duas coisas que já temos:

- $x :: l$ é uma permutação de $m :: r$;
- m é menor ou igual a todo mundo em $x :: l$.

Como uma permutação não muda quais elementos estão na lista (só a ordem), qualquer elemento de r também está na lista original. Então, se m era menor que todos os elementos, ele continua sendo menor que todo mundo em r .

Lemma *select_le_all_rest* :

$\forall x \ l \ m \ r,$
 $\text{select } x \ l = (m, r) \rightarrow$
 $\text{le_all } m \ r.$

7 Interface semelhante à função original `select_min`

A nossa nova função `select` devolve um par (m, r) para o Selection Sort continuar. Só que, para manter as coisas parecidas com o projeto original, que tinha uma função `select_min` que só devolvia o menor elemento, criamos essa função aqui de interface.

Ela utiliza o `select` por dentro. Como a lista pode vir vazia, usamos o tipo `option nat`:

- Ela devolve `None` se a lista estiver vazia.
- E `Some m` com o menor valor se tiver elementos.

Definition *select_min* ($l : \text{list nat}$) : *option nat* :=

`match l with`
`| []  $\Rightarrow$  None`
`| x :: xs  $\Rightarrow$  Some (fst (select x xs))`
`end.`

Aqui a gente prova que a nossa interface está certa, se ela devolver `Some m`, então esse m realmente é menor ou igual a todo mundo na lista l .

Lemma *select_min_correct* :

$\forall l \ m,$
 $\text{select_min } l = \text{Some } m \rightarrow$
 $\text{le_all } m \ l.$

8 Função principal do Selection Sort

O Coq/Rocq é meio difícil com funções recursivas, ele exige que a gente garanta que a função vai terminar e não vai ficar rodando para sempre. Como a nossa função `select` diminui a lista de um jeito que não é tão óbvio visualmente, a gente usa um parâmetro “combustível” (*fuel*).

Toda vez que a função faz uma chamada recursiva, ela gasta 1 unidade de combustível. Se o combustível chegar a 0, ela para. Na nossa função principal `ss`, passamos o próprio tamanho da lista como combustível

inicial. Como a cada passo do Selection Sort ordenamos um elemento, esse combustível é perfeito e suficiente para terminar o trabalho.

```

Fixpoint ss_fuel (fuel : nat) (l : list nat) : list nat :=
  match fuel with
  | 0 ⇒ []
  | S fuel' ⇒
    match l with
    | [] ⇒ []
    | x :: xs ⇒

      let '(m, r) := select x xs in

      m :: ss_fuel fuel' r
    end
  end.

Definition ss (l : list nat) : list nat :=
  ss_fuel (length l) l.

```

9 Lemas auxiliares para a prova da ordenação

Aqui provamos que, se sabemos que “*x* é menor ou igual a todo mundo em *l1*”, e *l1* é uma permutação de *l2*, então “*x* também é menor ou igual a todo mundo em *l2*”.

Lemma *le_all_perm* :

```

  ∀ x l1 l2,
    Permutation l1 l2 →
    le_all x l1 →
    le_all x l2.

```

Ultizamos o predicado *Sorted* para dizer que uma lista está ordenada. Para provar que *x* :: *l* está ordenada, precisamos mostrar que o *x* é menor ou igual à cabeça de *l*. A biblioteca chama isso de *HdRel le x l*. Esse lemma prova que, se o *x* é menor que todo mundo (*le_all x l*), ele com certeza é menor que a cabeça.

Lemma *le_all_HdRel* :

```

  ∀ x l,
    le_all x l →
    HdRel le x l.

```

10 Correção da função com combustível

Esta é segunda parte do trabalho é muito importante. O lemma *ss_fuel_correct* vai provar duas coisas juntas de uma vez só:

- Que a lista de saída está ordenada (*Sorted*);
- Que a lista de saída é uma permutação da original.

Fazemos essa prova usando indução no número *n*, que representa o combustível e, por tabela, o tamanho da lista, a hipótese *length l = n*.

Lemma *ss_fuel_correct* :

```

  ∀ n l,
    length l = n →
    Sorted le (ss_fuel n l) ∧
    Permutation l (ss_fuel n l).

```

11 Teorema final

A nossa função `ss l` chama o `ss_fuel` passando exatamente o `length l`. Como isso bate direitinho com o que o lemma `ss_fuel_correct` pede. A prova fecha por reflexividade.

Theorem *selectionsort_correct* :

$$\forall l, \\ \text{Sorted } le \ (ss \ l) \wedge \\ \text{Permutation } l \ (ss \ l).$$

12 Exemplos executáveis

Exemplos para testar.

Example *selection_sort_example* :

`ss [3; 1; 4; 1; 2] = [1; 1; 2; 3; 4].`

Repositório-base: https://github.com/flaviodemoura/selection_sort